



Underworld3 published in Journal of Open Source Software

Louis Moresi¹ , Julian Giordani² , and John Mansour³ 

¹Australian National University, ²University of Sydney, ³Monash University

DOI <https://doi.org/10.59350/a085z-qzh60>

Abstract

The aim of underworld3 is to provide strong support to users in developing sophisticated mathematical models, and provide the ability to interrogate those models during development and at run-time. Underworld3 encodes the mathematical structure of the equations it solves in symbolic form.

Keywords Underworld Code, Python/Jupyter

JOSS 10.21105/joss.07831

A recent paper in the Journal of Open Source Software describes the implementation details of Underworld3 and a brief motivation for the rewritten codebase (Moresi et al, 2025).

Underworld3 combines the power of the symbolic algebra package, `sympy` (Meurer et al, 2017), with the equation templating system in the parallel finite-element framework of `PETSc`, through `petsc4py` (Knepley et al, 2013, Dalcin et al, 2011).

Underworld3 has the same capacity to mix Eulerian (mesh-based) variables with fully-Lagrangian (particle) variables in forming the complex expressions used to describe conservation equations and material properties.

The end result is a flexible, symbolic geodynamics package with a very low computational overhead and excellent parallel performance (here are the [parallel scaling results](#)).

1. GITHUB RELEASES AND ZENODO ARCHIVES

All Underworld releases are available through GitHub and are reflected in a [zenodo archive](#). There is a master doi for all Underworld3 releases to allow you to cite the software as a project: Louis Moresi et al. (2026). Individual releases have their own (unique) doi.

2. EXCERPT — THE MATHEMATICAL FRAMEWORK OF UNDERWORLD 3

The symbolic layer of `underworld3` works with the “strong form” of a problem which is typically how the governing equations are derived and disseminated in publications and textbooks. The finite element method is based on a corresponding weak or variational form.

PETSc provides a template form for the automatic generation of weak forms [see Knepley et al, 2013]. We start from the strong-form of the problem which is defined through the functional $\mathcal{F}_s$ that expresses the balance between fluxes ($F(u, \nabla u)$), forces, $f(u, \nabla u)$, and unknowns u :

$$\mathcal{F}_s(u) \sim \nabla \cdot F(u, \nabla u) - f(u, \nabla u) = 0 \quad (1)$$

The discrete weak form and its Jacobian derivative would then be expressed through the related functional $\mathcal{F}_w$ as follows:

$$\mathcal{F}_w(u) \sim \sum_e \varepsilon_e^T \left[B^T W f(u^q, \nabla u^q) + \sum_k D_k^T W F^k(u^q, \nabla u^q) \right] = 0 \quad (2)$$

$$\mathcal{F}_w'(u) \sim \sum_e \varepsilon_e^T \begin{bmatrix} B^T & D^T \end{bmatrix} W \begin{bmatrix} \partial f / \partial u & \partial f / \partial \nabla u \\ \partial F / \partial u & \partial F / \partial \nabla u \end{bmatrix} \begin{bmatrix} B^T \\ D^T \end{bmatrix} \varepsilon_e \quad (3)$$

Here ε is the element restriction operator; B is the matrix of basis function derivatives and D is the constitutive matrix that, together, describe the relation between the unknowns and the flux. q indicates that the values are determined at a set of quadrature points, and W is a diagonal matrix of weights for these points.

The symbolic representation of the strong-form that is encoded in `underworld3` is:

$$\left[Du/Dt \right] - \nabla \cdot \left[\sigma(u, \nabla u, x, t) \right] - \left[H(u, \nabla u, x, t) \right] = 0 \quad (4)$$

Here H represents sources and sinks of u , and Du/Dt is the material time derivative of u . The time derivatives of the unknowns are not present in the PETSc template but, after time-discretisation, they produce terms that are combinations of fluxes and flux history terms (which combine with σ to contribute to F) and forces (which combine with h to contribute to f). The explicit time / position dependence in σ is to highlight potential changes to boundary conditions or constitutive properties.

In `underworld3`, the user interacts with the time derivatives explicitly, and provides strong-form expressions for the template in (4). Sympy automatically gathers all the flux-like terms and all the force-like terms into the form required by the PETSc template. All evaluations, derivatives and simplifications of functions in the `underworld3` symbolic layer are deferred until final assembly of the PETSc template and the compilation of the C functions.

The main benefits of combining `sympy` with the PETSc weak form template is a user environment that 1) provides symbolic, mathematical introspection, particularly in the context of Jupyter notebooks; 2) eliminates much of the `python` or `C` coding required for complex constitutive models; 3) eliminates any need for users to compute derivatives for the Newton solvers in PETSc.

3. REFERENCES

Dalcin, L. D., Paz, R. R., Kler, P. A., & Cosimo, A. (2011). Parallel distributed computing using Python. *Advances in Water Resources*, 34(9), 1124–1139. <https://doi.org/10.1016/j.advwatres.2011.04.013>

Knepley, M. G., Brown, J., Rupp, K., & Smith, B. F. (2013). Achieving High Performance with Unified Residual Evaluation. arXiv:1309.1204 [Cs]. <https://arxiv.org/abs/1309.1204>

Meurer, A., Smith, C. P., Paprocki, M., Čertík, O., Kirpichev, S. B., Rocklin, M., Kumar, A., Ivanov, S., Moore, J. K., Singh, S., Rathnayake, T., Vig, S., Granger, B. E., Muller, R. P., Bonazzi, F., Gupta, H., Vats, S., Johansson, F., Pedregosa, F., ... Scopatz, A. (2017). SymPy: Symbolic computing in Python. PeerJ Computer Science, 3, e103. <https://doi.org/10.7717/peerj-cs.103>

Moresi et al. (2025). Underworld3: Mathematically Self-Describing Modelling in Python for Desktop, HPC and Cloud. Journal of Open Source Software, 10(112), 7831. <https://doi.org/10.21105/joss.07831>

The Underworld project is supported by AuScope and the Australian Government through the National Collaborative Research Infrastructure Strategy (NCRIS). Source code: github.com/underworldcode/underworld3

REFERENCES

Louis Moresi, Julian Giordani, Matt Knepley, Romain Beucher, Thyagarajulu Gollapalli, Juan Carlos Graciosa, Ben Knight, Neng Lu, John Mansour, & Romain Beucher. (2026,). *Underworld3: Mathematically Self-Describing Modelling in Python for Desktop, HPC and Cloud*. Zenodo. <https://doi.org/10.5281/ZENODO.16810746>